

SupportOps AI

Full-Stack AI Orchestrator — Assignment + Dataset + UI Mockup Pack

A practical project for moving from full-stack engineering into production AI application engineering.

Core stack: React + TypeScript, Node.js + TypeScript, PostgreSQL + pgvector, LLM API, Zod, LangGraph, Langfuse, Docker.

Project objective

Build a customer-support and operations platform where an AI assistant can answer questions from company documents, inspect customer/order data, use backend tools, and request human approval before sensitive actions such as refunds or cancellations.

The supplied starter pack contains relational CSV data, RAG-ready policy PDFs, a data-ingestion guide, a roadmap, and frontend mockups.

1. Assignment Scope

- **LLM integration:** streaming chat, context, error handling and provider integration.
- **Structured output:** Zod validation for model-generated machine-readable data.
- **Tool calling:** customer/order/search/refund tools executed only by the backend.
- **RAG:** ingest the supplied PDFs, chunk them, embed them, store vectors in pgvector, retrieve sources.
- **Orchestration:** use LangGraph for state, routing, loops, retries and controlled execution.
- **Human approval:** server-enforced approval for sensitive state-changing actions.
- **Observability:** Langfuse traces for LLM calls, tools, latency and tokens.
- **Evaluation:** 30+ regression cases including prompt-injection attempts.
- **Security:** authorization, narrow tool scope, audit events, rate limiting and max agent steps.

2. Recommended Build Order

1. Normal application → 2. Basic LLM → 3. Structured outputs → 4. Tools → 5. RAG → 6. LangGraph → 7. Human approval → 8. Observability → 9. Evals → 10. Hardening.

This order is deliberate. Learn the primitives before hiding them behind an agent framework.

3. Supplied Dataset

The starter pack includes five CSV files:

- **customers.csv** — customer identity and support tier.
- **products.csv** — product catalog, pricing and stock.
- **orders.csv** — customer orders, status and delivery dates.
- **order_items.csv** — order/product relationships.
- **support_tickets.csv** — historical support cases.

And four PDF knowledge-base documents:

- **refund-policy.pdf**
- **shipping-policy.pdf**
- **warranty-policy.pdf**
- **faq.pdf**

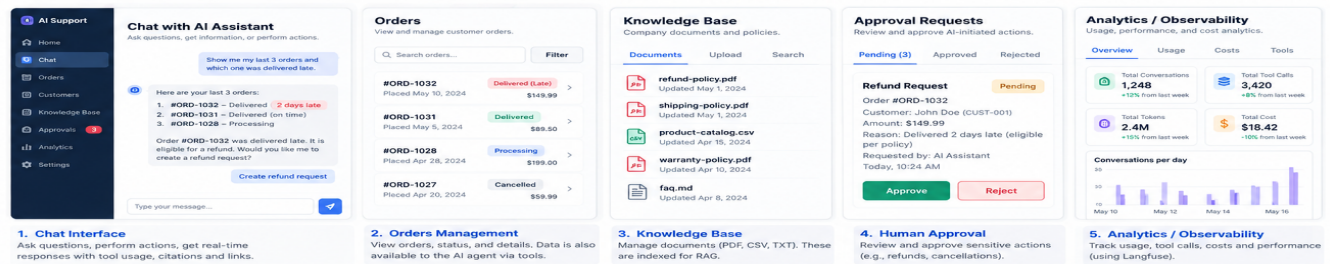
Use the CSVs for deterministic business data and the PDFs for RAG. This separation is intentional: the AI should retrieve policy knowledge while querying transactional facts through backend tools.

4. Frontend Mockups

Use the following mockups as a visual starting point. You do not need to reproduce them pixel-for-pixel; focus on the workflows and information architecture.

1. Frontend App Mockups (React)

A modern UI for an AI-powered support and operations platform.



2. Sample Data Files (for your database & RAG)

Use these sample files to populate your database and knowledge base.

customers.csv

id	name	email	tier	join_date
CUST-001	John Doe	john@example.com	gold	2023-06-12
CUST-002	Jane Smith	jane@example.com	silver	2023-08-23
CUST-003	Mike Johnson	mike@example.com	bronze	2024-01-15
CUST-004	Emily Davis	emily@example.com	gold	2023-11-02
CUST-005	David Wilson	david@example.com	silver	2024-02-18

orders.csv

id	customer_id	product_id	status	total_amount	order_date
ORD-1032	CUST-001	PROD-001	delivered	149.99	2024-05-10
ORD-1031	CUST-001	PROD-002	delivered	89.50	2024-05-05
ORD-1028	CUST-001	PROD-003	processing	199.00	2024-04-28
ORD-1027	CUST-002	PROD-004	cancelled	59.99	2024-04-20
ORD-1026	CUST-003	PROD-001	delivered	149.99	2024-04-18

products.csv

id	name	category	price	stock
PROD-001	Wireless Headphones	Electronics	149.99	120
PROD-002	Laptop Stand	Accessories	89.50	85
PROD-003	Mechanical Keyboard	Electronics	199.00	45
PROD-004	Mouse Pad	Accessories	59.99	200
PROD-005	USB-C Hub	Electronics	79.99	150

order_items.csv

id	order_id	product_id	quantity	unit_price
OI-1001	ORD-1032	PROD-001	1	149.99
OI-1002	ORD-1031	PROD-002	1	89.50
OI-1003	ORD-1028	PROD-003	1	199.00
OI-1004	ORD-1027	PROD-004	1	59.99
OI-1005	ORD-1026	PROD-001	1	149.99

refund-policy.pdf

Refund Policy

Last Updated: May 1, 2024

- Eligibility**
 - Orders can be refunded within 30 days of delivery.
 - Items must be in original condition.
 - Digital products are non-refundable.
- Late Delivery**

If an order is delivered more than 2 days after the estimated delivery date, customers are eligible for a full refund or 20% credit.

shipping-policy.pdf

Shipping Policy

Last Updated: May 1, 2024

- Shipping Times**
 - Standard Shipping: 3-5 business days
 - Express Shipping: 1-2 business days
 - International Shipping: 7-14 business days
- Delays**

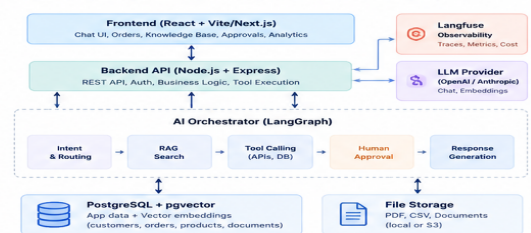
While we aim to meet estimated delivery dates, delays may occur due to weather, holidays, or carrier issues. Orders delayed more than 2 days are eligible for compensation.

product-catalog.csv (for RAG)

id	name	category	description
PROD-001	Wireless Headphones	Electronics	Noise-cancelling over-ear headphones...
PROD-002	Laptop Stand	Accessories	Adjustable aluminum laptop stand...
PROD-003	Mechanical Keyboard	Electronics	Compact mechanical keyboard with RGB...
PROD-004	Mouse Pad	Accessories	Large desk mouse pad with anti-slip base...
PROD-005	USB-C Hub	Electronics	7-in-1 USB-C hub with HDMI, USB-A...

3. System Architecture

Full-stack architecture with AI orchestration, RAG, tools, and observability.



4. Suggested Approach (Step by Step)

Follow this sequence to build and learn.

- Project Setup** — Initialize React frontend and Node.js backend. Set up PostgreSQL with Docker.
- Database** — Create schema and import CSV files (customers, orders, products, order_items).
- Basic LLM Integration** — Build a simple chat endpoint with streaming responses.
- Structured Outputs** — Use Zod to validate LLM responses.
- Tool Calling** — Implement tools (getOrders, getCustomer, createRefund, searchProducts, etc.).
- RAG Pipeline** — Parse PDF/CSV files, chunk, create embeddings, store in pgvector, and implement search.
- Agent Orchestration** — Build a LangGraph agent with routing, tool calling, and retries.
- Human Approval** — Add approval flow for sensitive actions (e.g., refunds, cancellations).
- Observability** — Integrate Langfuse for tracing, token usage, and analytics.
- Evaluation** — Create test cases and evaluate responses (promptfoo / custom script).

- **Chat:** conversation, tool activity, sources and action suggestions.
- **Orders:** searchable order list and detail view.
- **Knowledge Base:** document upload, indexing status and source search.
- **Approvals:** pending/approved/rejected sensitive actions.
- **Analytics:** agent runs, tool calls, tokens, cost and latency.

5. Target Architecture

React UI



Node.js API → authentication / authorization / business logic



AI Orchestrator (LangGraph)



RAG search LLM Backend tools



PostgreSQL + pgvector Orders / customers / tickets

Langfuse observes the AI workflow; audit events record sensitive operations.

6. Data Ingestion Approach

Start simple:

- Seed the CSV data directly into PostgreSQL with a Node/TypeScript seed script.
- Extract text from the supplied PDFs using a PDF parser.
- Store original documents and metadata.
- Split text into chunks.
- Generate embeddings.
- Store embeddings in pgvector.
- Retrieve the top relevant chunks for each knowledge question.
- Return source metadata to React so the user can see where the answer came from.

Do not build a generalized ingestion platform on day one. First make the end-to-end RAG workflow work with these four PDFs, then generalize ingestion to uploaded documents.

7. Suggested Database Groups

Business tables

- users, customers, products, orders, order_items, support_tickets

AI/RAG tables

- knowledge_documents, knowledge_chunks, agent_runs, tool_calls

Safety/audit tables

- approval_requests, audit_events

8. Initial Backend Tools

- getCustomer(customerId)
- getOrders(customerId)
- getOrder(orderId)
- getShipmentStatus(orderId)
- searchProducts(query)
- searchKnowledge(query)
- checkRefundEligibility(orderId)
- createRefund(orderId) — approval required
- cancelOrder(orderId) — approval required

Every tool should have a strict input schema, authorization check, bounded database query, timeout/error handling, and an audit record where appropriate.

9. Acceptance Criteria

- React chat streams responses.
- The agent can answer order questions from PostgreSQL data.
- Policy questions retrieve relevant PDF content and display sources.
- The agent can perform multi-step tool workflows.
- Refunds/cancellations require backend-enforced human approval.
- Sensitive actions are auditable.
- Langfuse or an equivalent traces the AI workflow.
- At least 30 evaluation cases run repeatably.
- Prompt injection cannot bypass authorization.
- The complete system runs locally with documented setup commands.

10. How to Use the Dataset While Learning

A good progression through the supplied data is:

- **Step A:** Import customers/products/orders/order_items. Build normal REST endpoints and React pages.
- **Step B:** Ask the LLM questions, but initially give it no tools.
- **Step C:** Add getOrders and getOrder. Notice how tool calling turns the LLM into an interface to your existing backend.
- **Step D:** Add the policy PDFs. Build searchKnowledge and expose citations.

- **Step E:** Combine RAG and tools in multi-step questions.
- **Step F:** Add checkRefundEligibility and then createRefund behind approval.
- **Step G:** Trace everything and start measuring failures.

11. Example End-to-End Scenario

User: "My latest order arrived late. Am I eligible for compensation? If so, refund it."

Expected conceptual flow:

1. Agent calls getOrders(customer).
2. Agent identifies latest delivered order.
3. Agent calls getOrder(orderId).
4. Agent retrieves refund policy through RAG.
5. Agent calls checkRefundEligibility(orderId).
6. If eligible, agent creates an ApprovalRequest rather than directly refunding.
7. Human approves in React.
8. Backend re-checks authorization and eligibility.
9. createRefund executes.
10. AuditEvent is stored and the assistant reports the result.

12. Stretch Goals

- Background jobs with BullMQ, Inngest, Trigger.dev or Temporal.
- Generic file-upload ingestion pipeline.
- Multiple LLM provider abstraction.
- Semantic caching and cost budgets.
- Automated evals in CI.
- Admin dashboard for traces and approval history.
- Model fallback and resilience strategies.

13. Final Engineering Goal

By completing this project, you should be able to explain and implement the boundary between deterministic software and probabilistic AI. The transferable skill is not memorizing an agent framework; it is designing safe, observable, testable AI-powered software around your existing full-stack engineering foundation.